



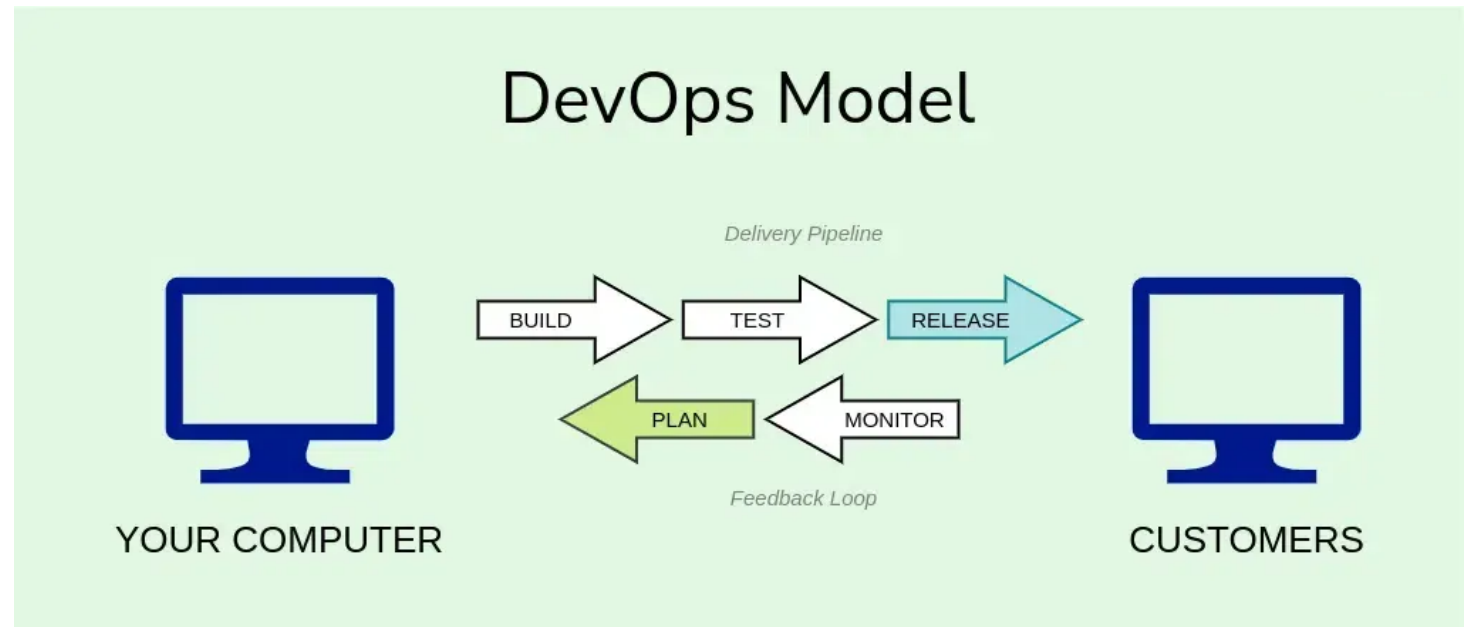
FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE 12 - DEVOPS PRACTICES



WHAT IS DEVOPS?

- DevOps is all about automating and streamlining the software development lifecycle so that code moves from development to production quickly, reliably, and securely.



WHAT IS DEVOPS?

- **Stages of DevOps are:**

- **Build Stage**

1. Developers **write and organize code**, using version control tools like Git to track changes.
2. The system **automatically compiles and packages the code** into a deployable format.
3. Dependencies (external libraries and tools) are included to ensure smooth operation.
4. Common Tools: Git, Jenkins, GitLab CI/CD, Gradle, Maven.

- **Test Stage**

1. The software undergoes **thorough testing** to catch bugs and security risks before release.
2. Different testing methods include:
 - **Unit Testing:** Checks individual pieces of code.
 - **Integration Testing:** Ensures different parts of the system work together.
 - **Performance Testing:** Measures speed and scalability.
 - **Security Testing:** Identifies potential vulnerabilities.
3. Automated tests help ensure the software is stable before moving forward.
4. Common Tools: Selenium, JUnit, TestNG, SonarQube.

WHAT IS DEVOPS?

■ Release Stage

1. The software is deployed in a **staging environment** to simulate real-world conditions.
2. If everything checks out, the software is **rolled out to production** using deployment strategies like:
 - **Blue-Green Deployment:** Two identical environments switch traffic for a seamless update.
 - **Canary Deployment:** A small percentage of users get the new version first, ensuring safety.
 - **Rolling Updates:** The update is gradually pushed out to all users.
3. Common Tools: Docker, Kubernetes, Ansible, Helm, ArgoCD.

■ Continuous Feedback Loop

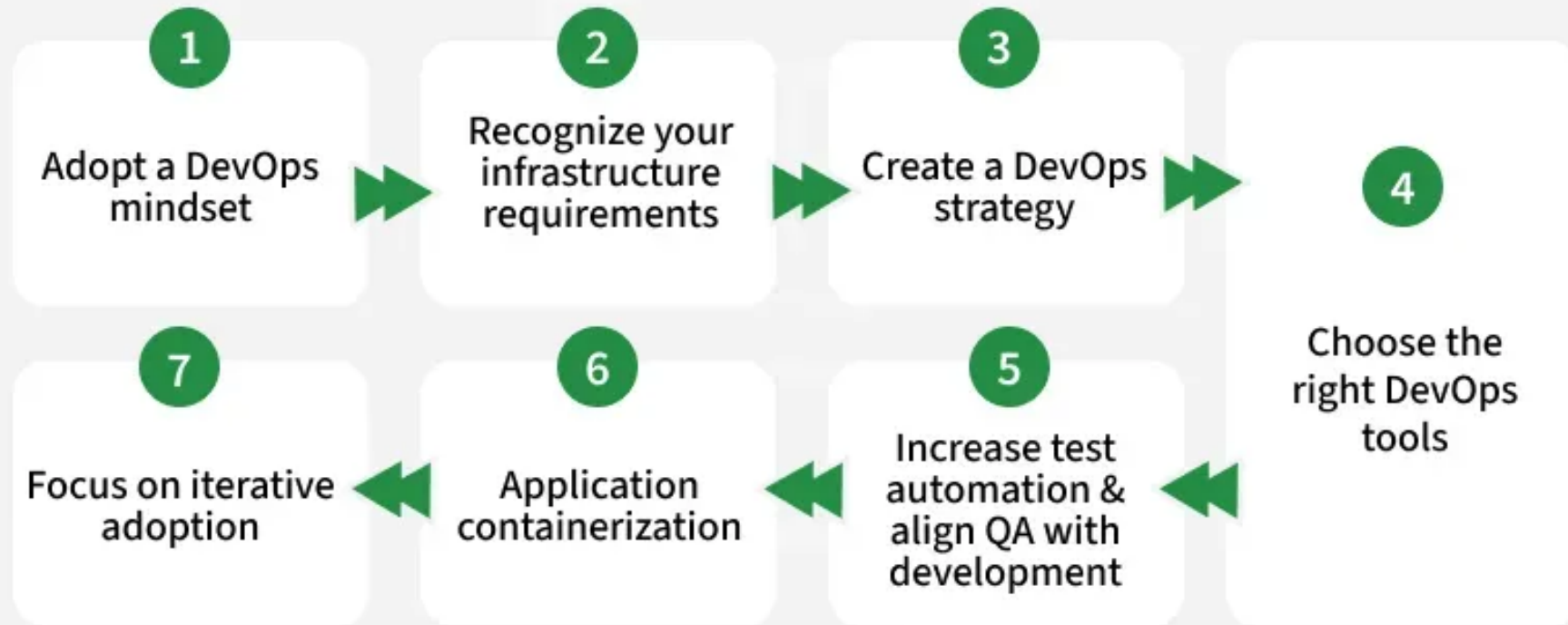
- A key aspect of DevOps is **learning from real-world performance** and using that feedback to improve future releases.
 1. **Monitoring & Logging:** Track system performance and detect errors.
 2. **User Feedback:** Gather insights from customers to enhance features.
 3. **Incident Response:** Alert systems notify teams of failures for quick fixes.
 4. **Process Improvement:** Teams analyze past releases to optimize automation and workflow.
 5. **Common Tools:** Prometheus, Grafana, ELK Stack, Datadog, New Relic.

WHAT IS DEVOPS?

- To adopt a DevOps model, ensure the following points:
 - **Assess Current Workflow:** Evaluate your existing development and operations processes to identify gaps, inefficiencies, and areas for automation.
 - **Set Clear DevOps Goals:** Define measurable objectives such as faster deployment cycles, better collaboration, or improved system stability.
 - **Build a Collaborative Culture:** Break the wall between development, operations, QA, and security teams by encouraging communication and shared responsibilities.
 - **Automate Infrastructure and Testing:** Use tools like Jenkins, Docker, and Ansible to automate code integration, deployment, testing, and infrastructure provisioning.
 - **Implement CI/CD Pipelines:** Establish Continuous Integration and Continuous Deployment pipelines for faster, error-free code delivery.
 - **Monitor and Optimize Continuously:** Use real-time monitoring and feedback loops (via tools like Prometheus, Grafana) to track performance and improve systems iteratively.

WHAT IS DEVOPS?

7 Steps to Successful DevOps Adoption



DEVOPS LIFECYCLE

- The **DevOps lifecycle** is a structured approach that integrates development (Dev) and operations (Ops) teams to streamline software delivery. It focuses on collaboration, automation, and continuous feedback across key phases planning, coding, building, testing, releasing, deploying, operating, and monitoring executed in a continuous loop.

DEVOPS LIFECYCLE

■ Phases of DevOps Lifecycle

- **Plan:** This phase focuses on understanding the business needs and gathering feedback from end-users. Teams create a plan that aligns the project with business goals and ensures the right results are delivered.
- **Code:** In this phase, developers write the actual code for the software. Tools like Git help manage the code, making sure that the code is well-organized and free from security issues or bad coding practices.
- **Build:** Once the code is written, it is submitted to a central system using tools like Jenkins. This step ensures the code is compiled, and all components are integrated together smoothly.
- **Test:** The software is then tested to ensure it works properly. This includes different types of tests like security, performance, and user acceptance. Tools like JUnit and Selenium are used to automate these tests and verify the software's integrity.
- **Release:** After testing, the software is ready to be released to production. The DevOps team ensures that all checks are passed and then sends the latest version to the production environment.
- **Deploy:** Using Infrastructure-as-Code (IaC) tools like Terraform, the necessary infrastructure (servers, networks, etc.) is automatically created. Once the infrastructure is set up, the code is deployed to various environments in an automated and repeatable way.
- **Operate:** Once deployed, the software is available for users. Tools like Chef help manage the configuration and ongoing deployment of the system to ensure it operates smoothly.
- **Monitor:** This phase involves observing how the software is performing in the real world. Data about user behaviour and application performance is collected to identify any issues or bottlenecks. By monitoring the system, the team can quickly spot and fix problems that may affect performance.

DEVOPS LIFECYCLE

■ 7 Cs of DevOps

- The 7 Cs of DevOps are core principles that help make DevOps successful. They guide how teams work together, build, test, and deliver software faster and more reliably. Each of these Cs contributes to a workflow that enhances the quality, speed, and reliability of delivering software products:

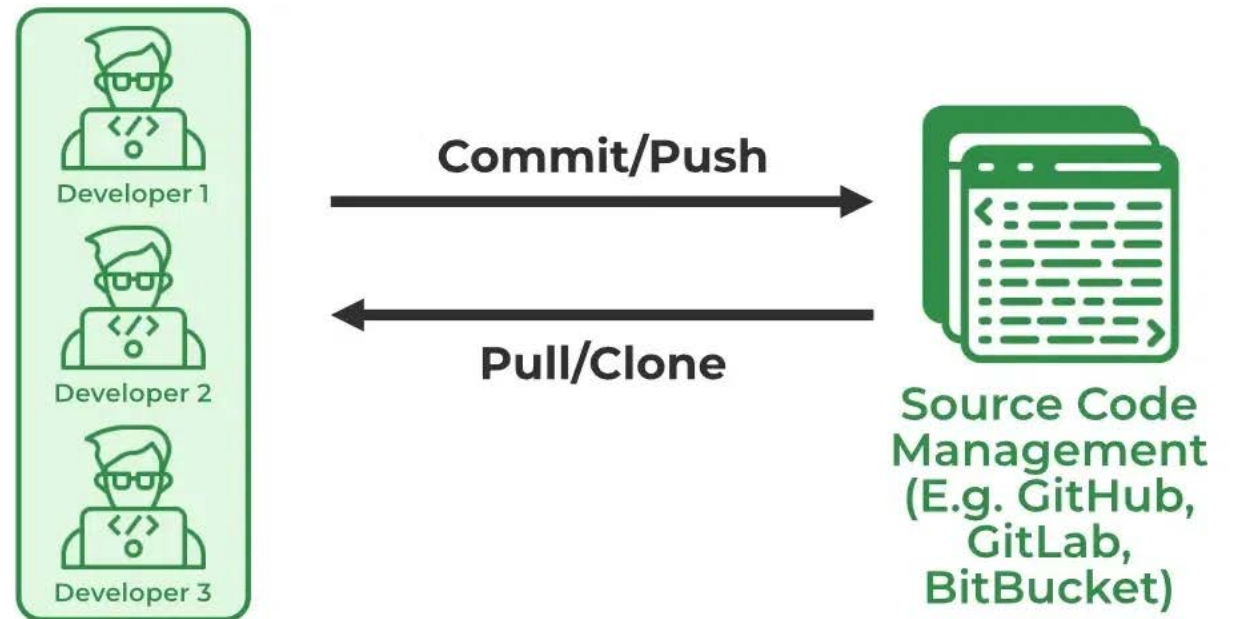
1. Continuous Development
2. Continuous Integration
3. Continuous Testing
4. Continuous Deployment/Continuous Delivery
5. Continuous Monitoring
6. Continuous Feedback
7. Continuous Operations



DEVOPS LIFECYCLE

I. Continuous Development

- Continuous Development involves the iterative and incremental approach to software creation, where development teams plan, code, and prepare software features in small, manageable units. This methodology enables rapid feedback, early detection of issues, and swift delivery of value to end-users. It integrates closely with version control systems and automation tools to streamline the development process.

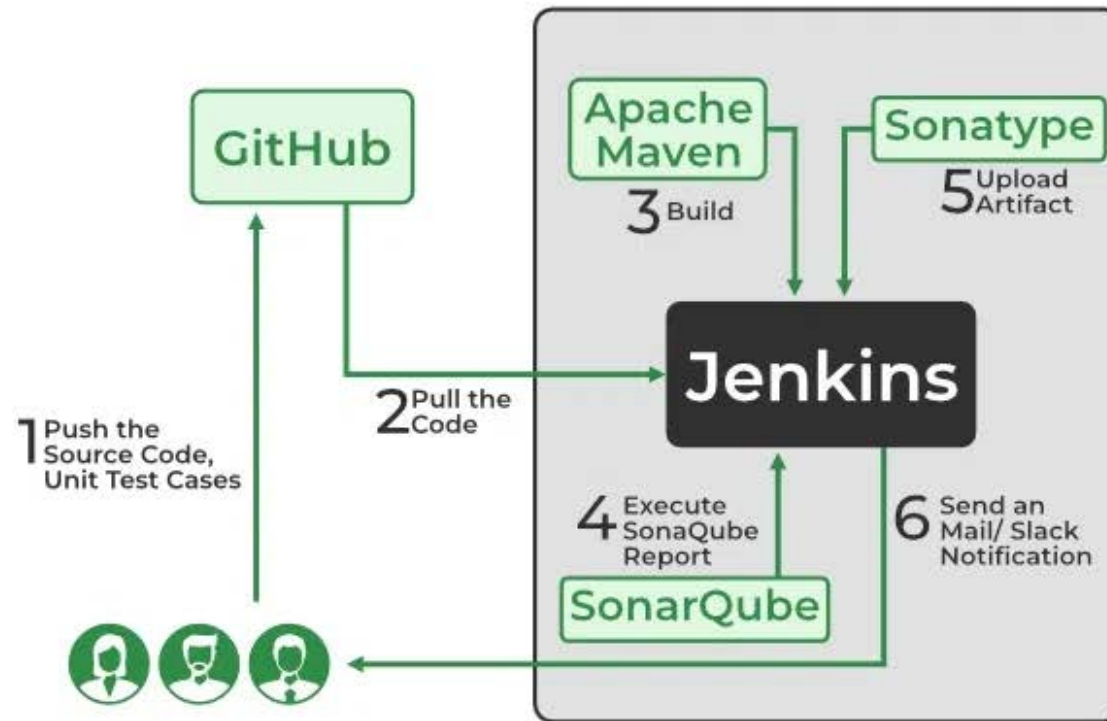


DEVOPS LIFECYCLE

2. Continuous Integration

- Continuous Integration (CI) in DevOps ensures that code changes made by developers are automatically built, tested, and integrated into the main codebase. This process typically involves four key stages:
 - **Source Code Management (SCM):** Developers push their code from local machines to a remote repository such as GitHub. This allows teams to collaborate, review, and manage code versions easily.
 - **Build Process:** The source code is then compiled using tools like Maven, which packages the application into artifacts such as .jar, .war, or .ear files.
 - **Code Quality Check:** Tools like SonarQube analyze the code for bugs and security issues. It generates detailed reports (HTML or PDF) to maintain code quality standards.
 - **Artifact Repository:** The generated build artifacts are stored in a repository manager like Nexus, which serves as a central storage for future deployment.
 - All these steps are automated using **Jenkins**, a popular CI tool that orchestrates the complete flow, from fetching code to storing the final build artifact.

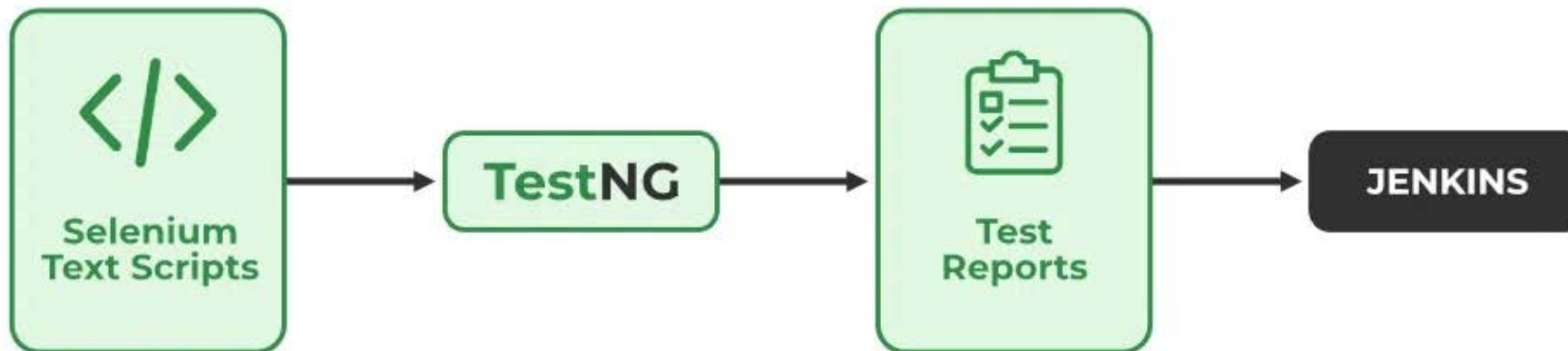
DEVOPS LIFECYCLE



DEVOPS LIFECYCLE

3. Continuous Testing

- **Continuous Testing** means testing the code automatically every time there is a change. This helps catch bugs early before the app goes live. With DevOps and Agile methods, companies can use tools like **Selenium**, **Testsigma**, or **LambdaTest** to test their applications automatically. These tools run tests faster and smarter than manual testing.
- Using a tool like **Jenkins**, we can set up the entire testing process to run **automatically** after every code change. This saves time and reduces human errors.



DEVOPS LIFECYCLE

4. Continuous Deployment/ Continuous Delivery

- **Continuous Deployment:** Continuous Deployment is the process of automatically deploying an application into the production environment when it has completed testing and the build stages. Here, we'll automate everything from obtaining the application's source code to deploying it.

Continuous Deployment



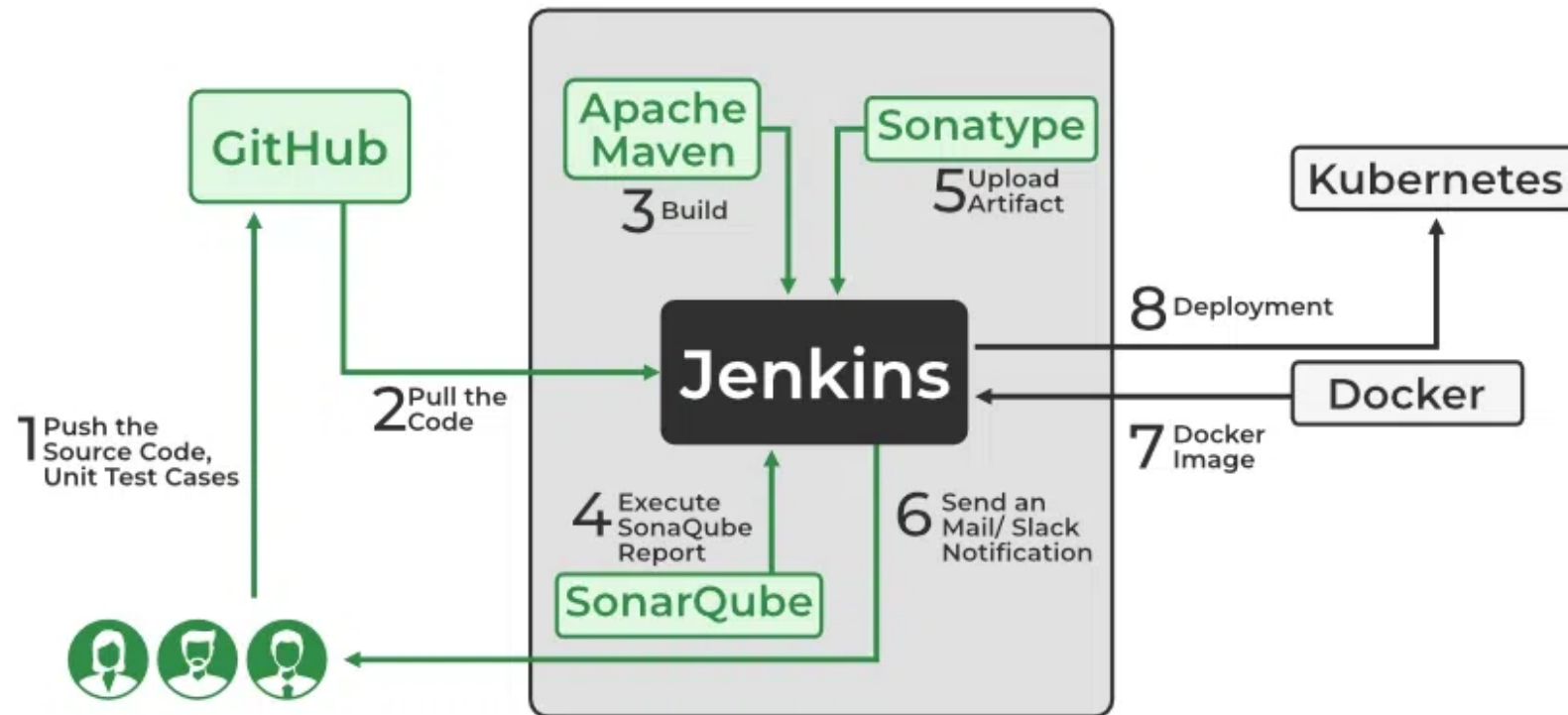
DEVOPS LIFECYCLE

- **Continuous Delivery:** Continuous Delivery is the process of deploying an application into production servers manually when it has completed testing and the build stages. Here, we will automate the continuous integration processes, however, manual involvement is still required for deploying it to the production environment.

Continuous Delivery



DEVOPS LIFECYCLE



DEVOPS LIFECYCLE

5. Continuous Monitoring

- The DevOps lifecycle is incomplete without Continuous Monitoring. Using tools like Prometheus and Grafana, teams can monitor systems in real time and receive alerts before issues occur. Prometheus collects a wide range of performance metrics - such as CPU and memory usage, network traffic, application response times, and error rates. Grafana then visualizes this time-series data, making it easier to observe trends, identify anomalies, and track overall system health.

6. Continuous Feedback

- Once the application is released, end users begin interacting with it and providing feedback regarding performance, bugs, and overall user experience. The DevOps team analyzes this feedback and works with the development team to address issues in the affected code. This continuous flow of information helps reduce bugs, improve quality, and streamline the deployment process. Ultimately, Continuous Feedback enhances application performance and ensures a smoother experience for end users.

7. Continuous Operations

- Continuous Operations focuses on maintaining high application uptime and minimizing maintenance-related downtime, which can negatively affect users. By implementing continuous operational practices, organizations can improve productivity, reduce operational costs, and achieve better quality control - resulting in more reliable and consistent service delivery.

POPULAR DEVOPS LIFECYCLE TOOLS

1. Plan

- Tools: Jira, Trello, Asana
- Used for task planning, assignments, and progress tracking.

2. Develop

- Tools: Git, GitHub, GitLab, Bitbucket
- Enables version control, code collaboration, and branching.

3. Build

- Tools: Jenkins, Maven, Gradle
- Automates the process of compiling code and managing dependencies.

4. Test

- Tools: Selenium, JUnit, TestNG, SonarQube
- Conducts automated testing for bugs, code quality, and security vulnerabilities.

POPULAR DEVOPS LIFECYCLE TOOLS

5. Release/Deploy

- Tools: ArgoCD, GitLab CI/CD, AWS CodeDeploy, Azure DevOps, Spinnaker, Terraform
- Automates deployment pipelines and software releases.

6. Operate

- Tools: Terraform, Ansible, Puppet, Chef
- Handles infrastructure provisioning and configuration management.

7. Monitor

- Tools: Prometheus, Grafana, ELK Stack, Datadog
- Tracks performance, logs, metrics, and system health.

INTRODUCTION TO GIT

- Git is a distributed version control system (VCS) used to track changes in source code during software development. It helps developers collaborate, manage different versions of code, and roll back to previous states if needed.
 - **Collaboration:** Multiple developers can work together and merge changes easily.
 - **History Tracking:** Revert to previous versions whenever needed.
 - **Branching & Merging:** Develop features separately and merge them safely.
 - **Distributed Development:** Each developer has a full copy of the repo.
 - **Incremental Changes:** Only differences are stored, saving space and simplifying updates.



Open source and free



Public and private repository hosting



Can work as a standalone program, a client, or a server



Distributed architecture

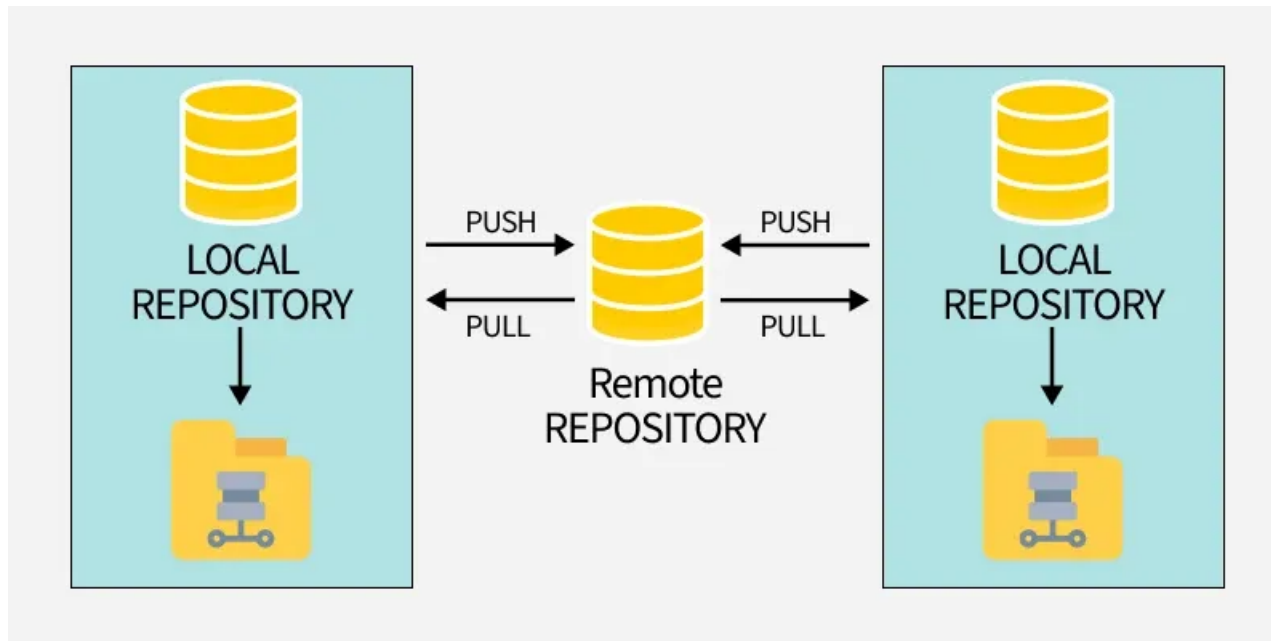


Fast and efficient



Does not depend on a centralized server

INTRODUCTION TO GIT



- Core concepts
 1. **Repositories**
- A repository (or repo) is a storage space where your project files and their history are kept. There are two types of repositories in Git:
 - **Local Repository:** A copy of the project on your local machine.
 - **Remote Repository:** A version of the project hosted on a server, often on platforms like GitHub, GitLab, or Bitbucket.

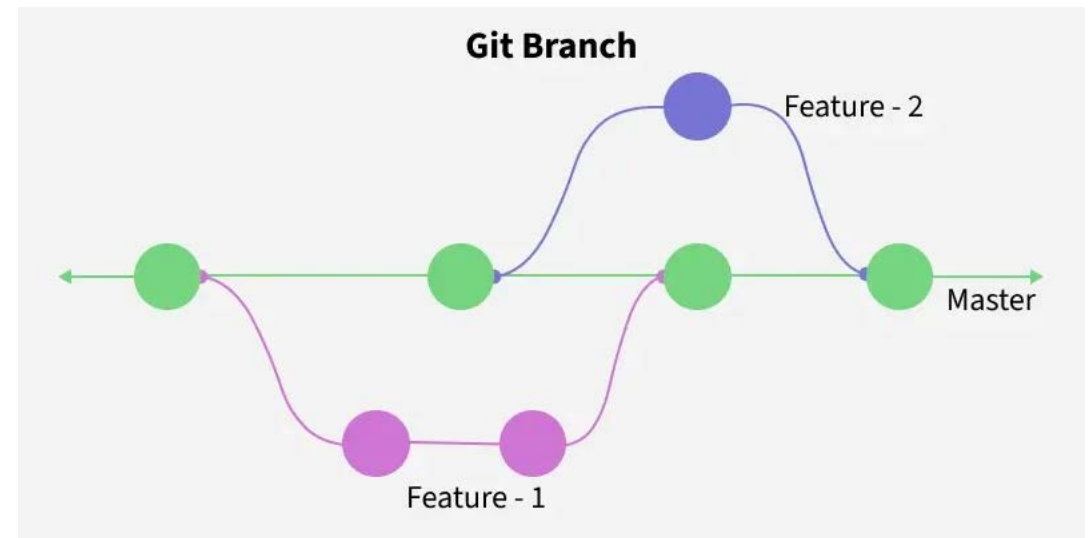
INTRODUCTION TO GIT

2. Commits

- A **commit** is a snapshot of your project at a specific point in time. Each commit has a unique identifier (hash) and includes a message describing the changes made. Commits allow you to track and review the history of your project.

3. Branches

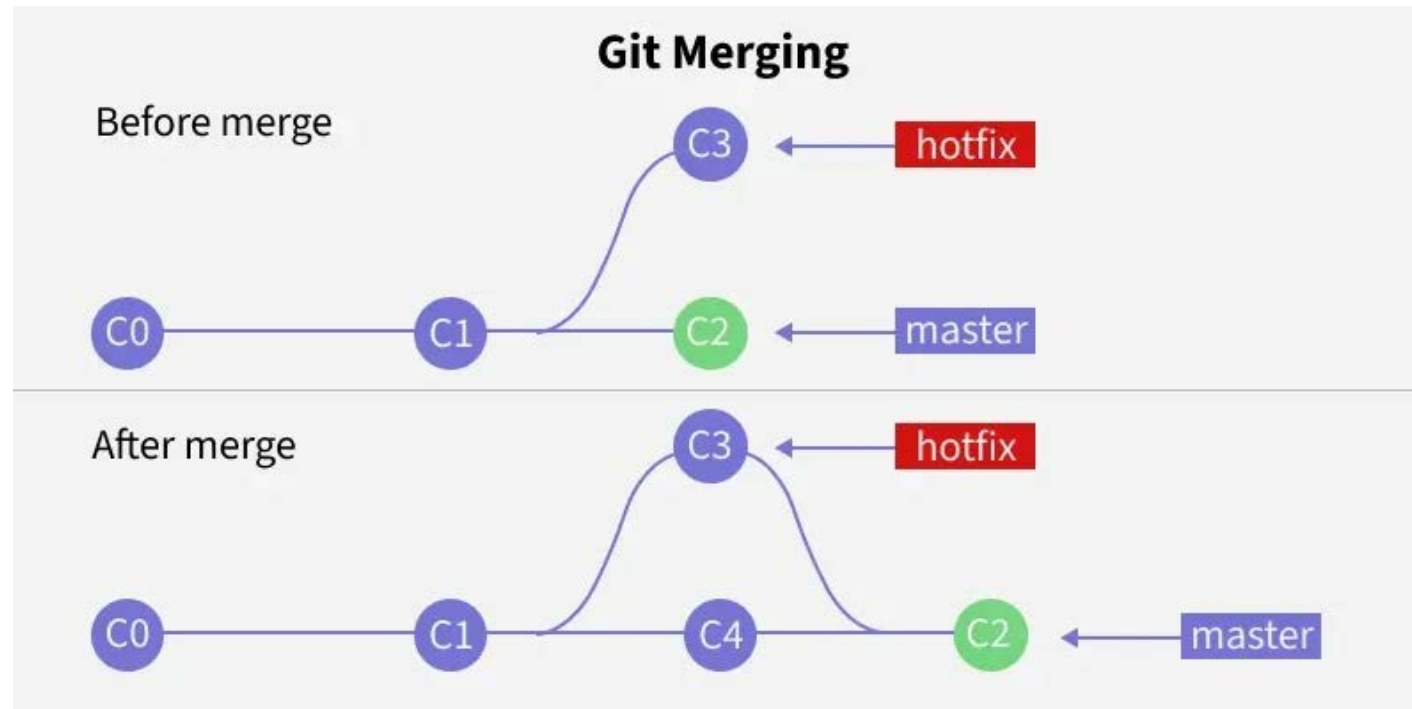
- Branches allow developers to work on separate tasks without affecting the main codebase. Common branch types include:
 - **Main (or Master) Branch:** The stable version of the project, usually production-ready.
 - **Feature Branch:** Used for developing new features or bug fixes.



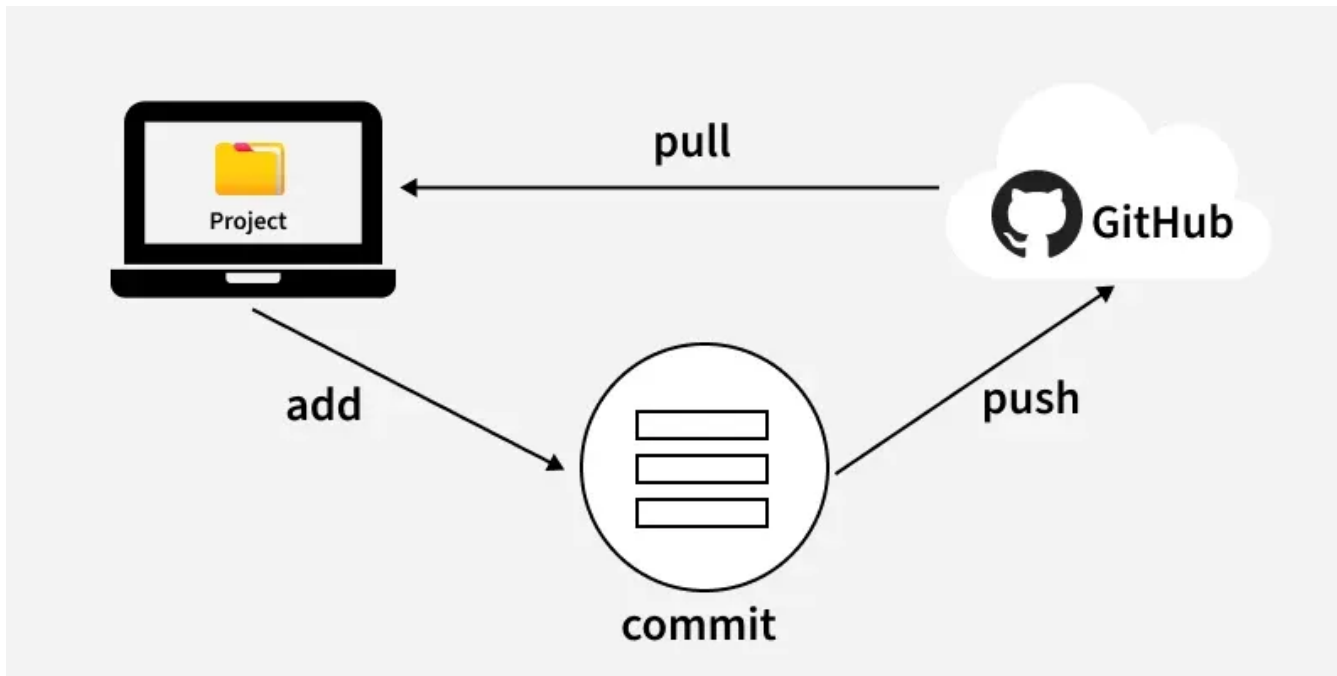
INTRODUCTION TO GIT

4. Merging

- Merging is the process of integrating changes from one branch into another. It allows you to combine the work done in different branches and resolve any conflicts that arise.



INTRODUCTION TO GIT



5. Cloning

- **Cloning** a repository means creating a local copy of a remote repository. This copy includes all files, branches, and commit history.

6. Pull and Push

- **Pull:** Fetches updates from the remote repository and integrates them into your local repository.
- **Push:** Sends your local changes to the remote repository, making them available to others.

INTRODUCTION TO GIT

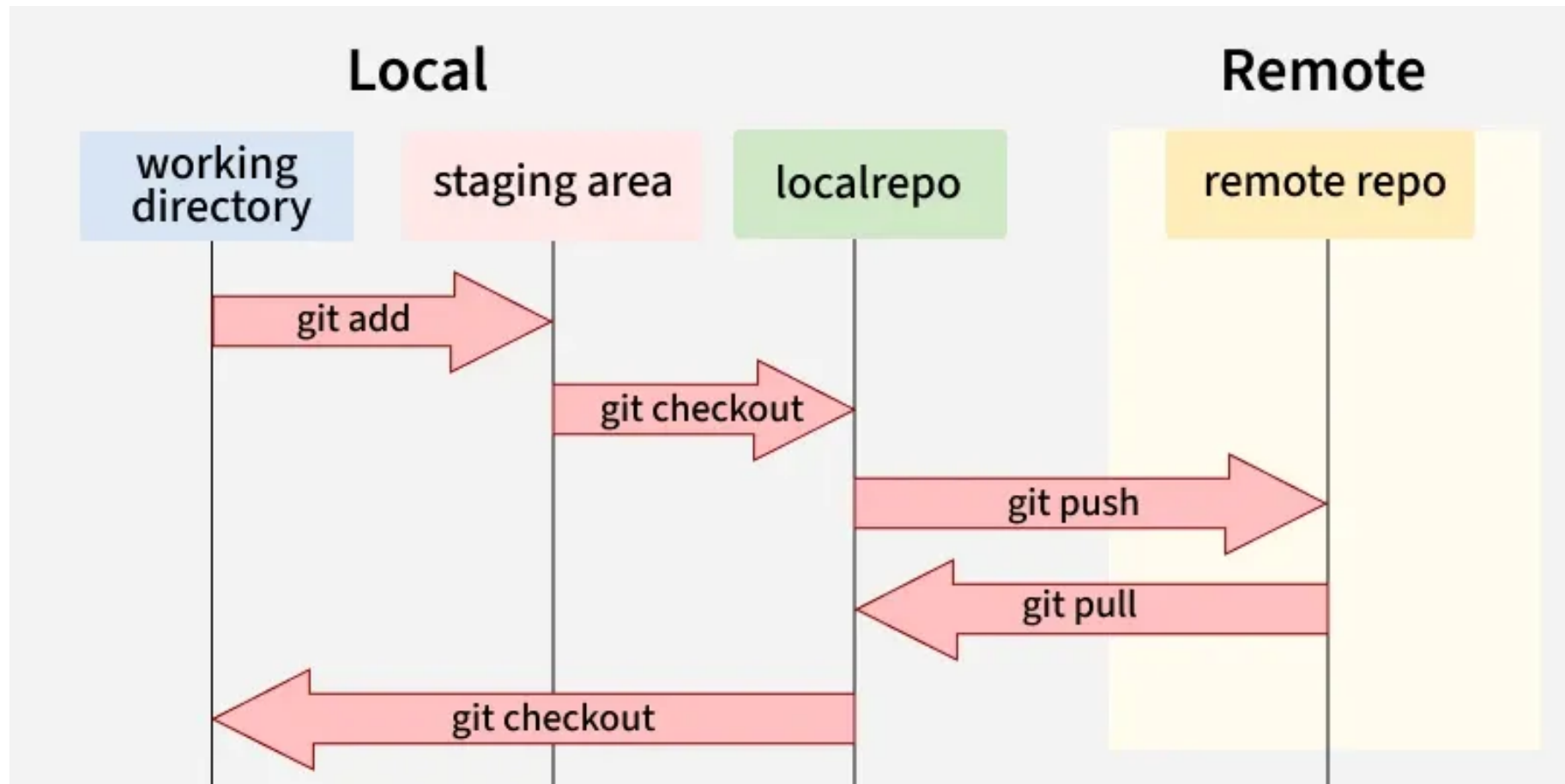
- In Git, your files can be in one of three main states. Understanding this is key to understanding how Git works:
 - **Working Directory:** This is your project folder with all the files you are currently working on. Any changes you make here are not yet tracked by Git.
 - **Staging Area (or Index):** This is an intermediate area where you list the specific changes you want to include in your next "snapshot" or commit. You use the `git add` command to move changes from your working directory to the staging area.
 - **Repository (.git directory):** This is where Git permanently stores the snapshots (commits) of your project. You use the `git commit` command to save the staged changes into the repository.
- This three-step process (**modify --> add --> commit**) gives you precise control over what gets saved in your project's history.

INTRODUCTION TO GIT

- Git commands are important for navigating and controlling your project repository. These commands help you manage files, track changes, and collaborate with others.

Command	Description
<code>git status</code>	Shows the current status of the repository - staged, unstaged, and untracked files.
<code>git add <file-name></code>	Stages a specific file for commit. Use <code>git add .</code> to stage all changes.
<code>git commit -m "message"</code>	Commits the staged changes with a descriptive commit message.
<code>git branch <branch-name></code>	Creates a new branch with the given name.
<code>git checkout <branch-name></code>	Switches to the specified branch.
<code>git merge <branch-name></code>	Merges changes from the given branch into the current branch.
<code>git push origin <branch-name></code>	Pushes the local branch changes to the remote repository.
<code>git pull origin <branch-name></code>	Fetches and merges changes from the remote repository into the local branch.
<code>git log</code>	Displays the commit history for the current branch.

INTRODUCTION TO GIT



INTRODUCTION TO GIT

1. Clone the Repository

```
git clone git@github.com:username/repository.git
```

2. Create and Switch to a New Branch

```
git checkout -b feature-branch
```

3. Make Changes and Stage Them

```
git add <file-name>
```

4. Commit the Changes

```
git commit -m "Add new feature"
```

5. Push the Changes

```
git push origin feature-branch
```

6. Create a Pull Request: After pushing your changes, create a pull request on GitHub to merge the feature branch into the main branch.

7. Update your Local Repository

```
git checkout main  
git pull origin main
```

8. Delete Feature Branch:

```
git branch -d feature-branch  
git push origin --delete feature-branch
```

INTRODUCTION TO GIT

- **Git Hosting**

- Git Hosting stores your Git repositories on a remote server, enabling collaboration, backup, pull requests, CI/CD, and even website hosting, while Git manages changes locally.

- 1. **GitHub:**

GitHub is a cloud-based Git hosting platform owned by Microsoft, popular for open-source projects and beginner developers. It provides a complete environment for version control, collaboration, and deployment.

- Unlimited public and private repositories.
- Pull requests and code reviews for team collaboration.
- GitHub Pages for hosting static websites.
- Automation and CI/CD with GitHub Actions.

- 2. **GitLab:**

- GitLab is a complete DevOps platform offering Git repository hosting along with built-in CI/CD, issue tracking, and project management tools. It is ideal for teams and enterprises looking to automate workflows and manage code efficiently.

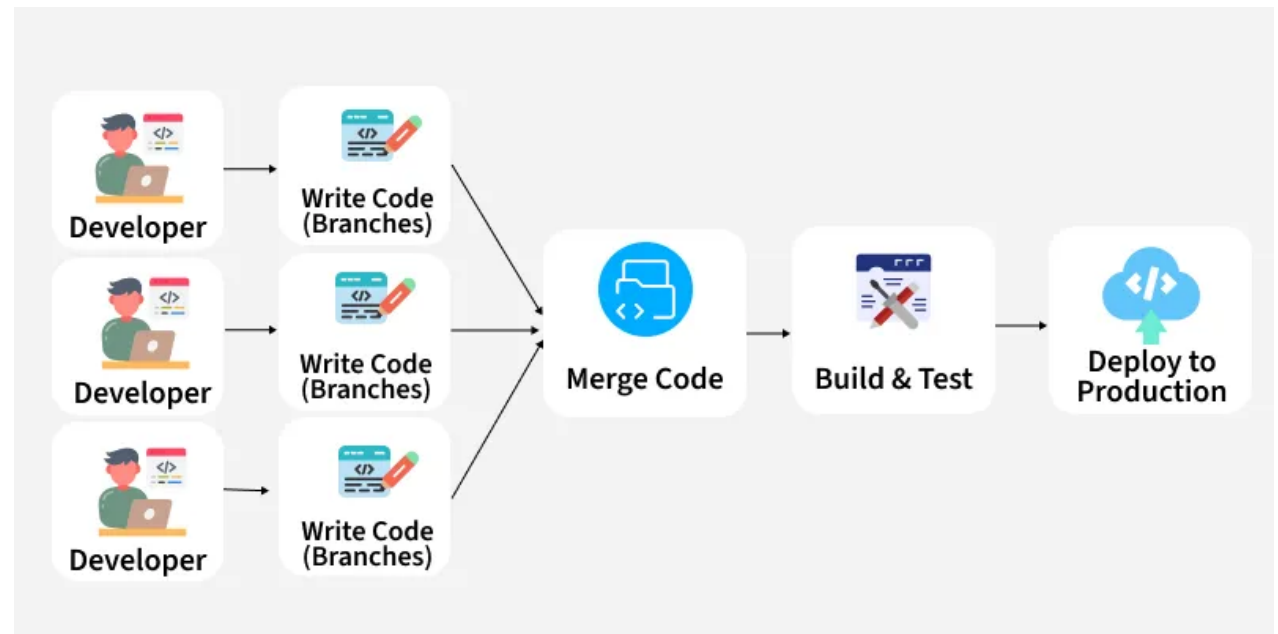
- Public and private repositories.
- Built-in CI/CD pipelines for automation.
- Issue tracking and project management.
- Option for self-hosting for full control and privacy.

WHAT IS CI/CD?

- CI/CD stands for Continuous Integration and Continuous Delivery/Deployment. With CI/CD, we automate the integration of code changes from multiple developers into a single codebase. It is a software development practice where the developers commit their work frequently to the central code repository (GitHub or Stash).
- **Continuous Integration:**
 - Continuous Integration is about how developers integrate code using a shared repository multiple times a day with the help of automation.
- **Continuous Delivery:**
 - Continuous Delivery is about automatically releasing software to the test or production environment.

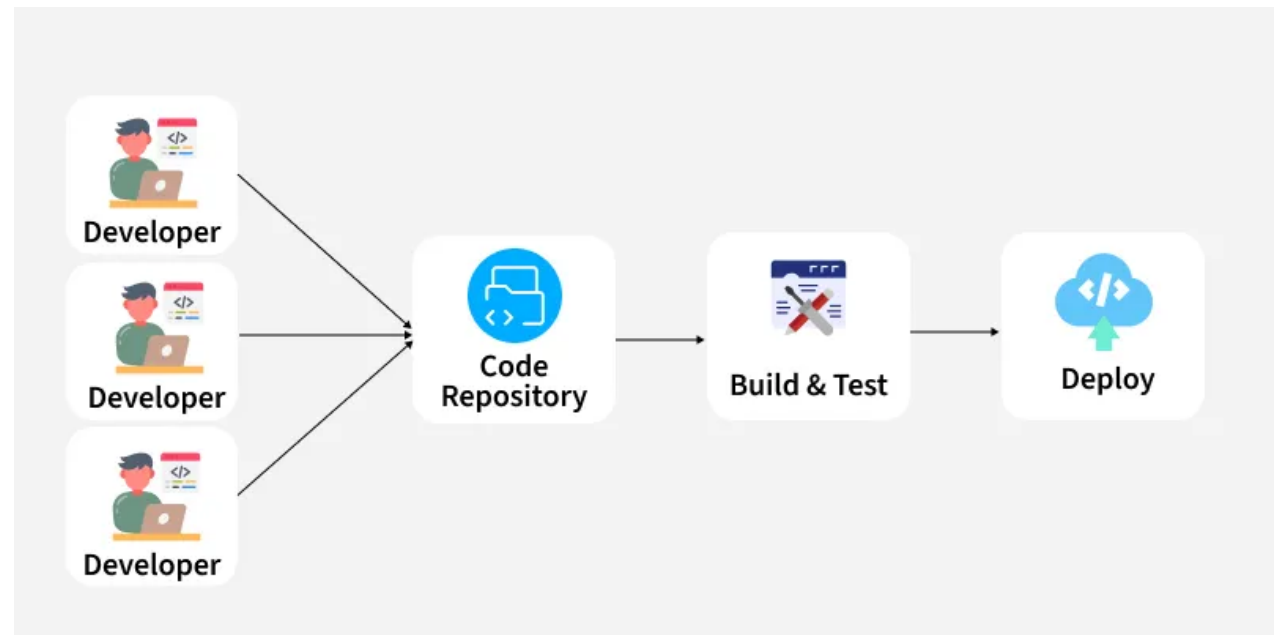
WHAT IS CI/CD?

- **Before CI/CD**
- In the past, software delivery was slow and full of manual work. Developers used to create separate feature branches and worked on them for weeks or even months.
- All these branches were merged at the end, which often caused big conflicts and broken builds. Testing and building the code was also manual, usually done at the final stage, so bugs were discovered very late and were costly to fix.
- Deployment was a long and risky process, taking days or sometimes weeks, because everything was released together in one big update.
- Teams were also divided: developers wrote code, testers tested it, and operations managed servers.

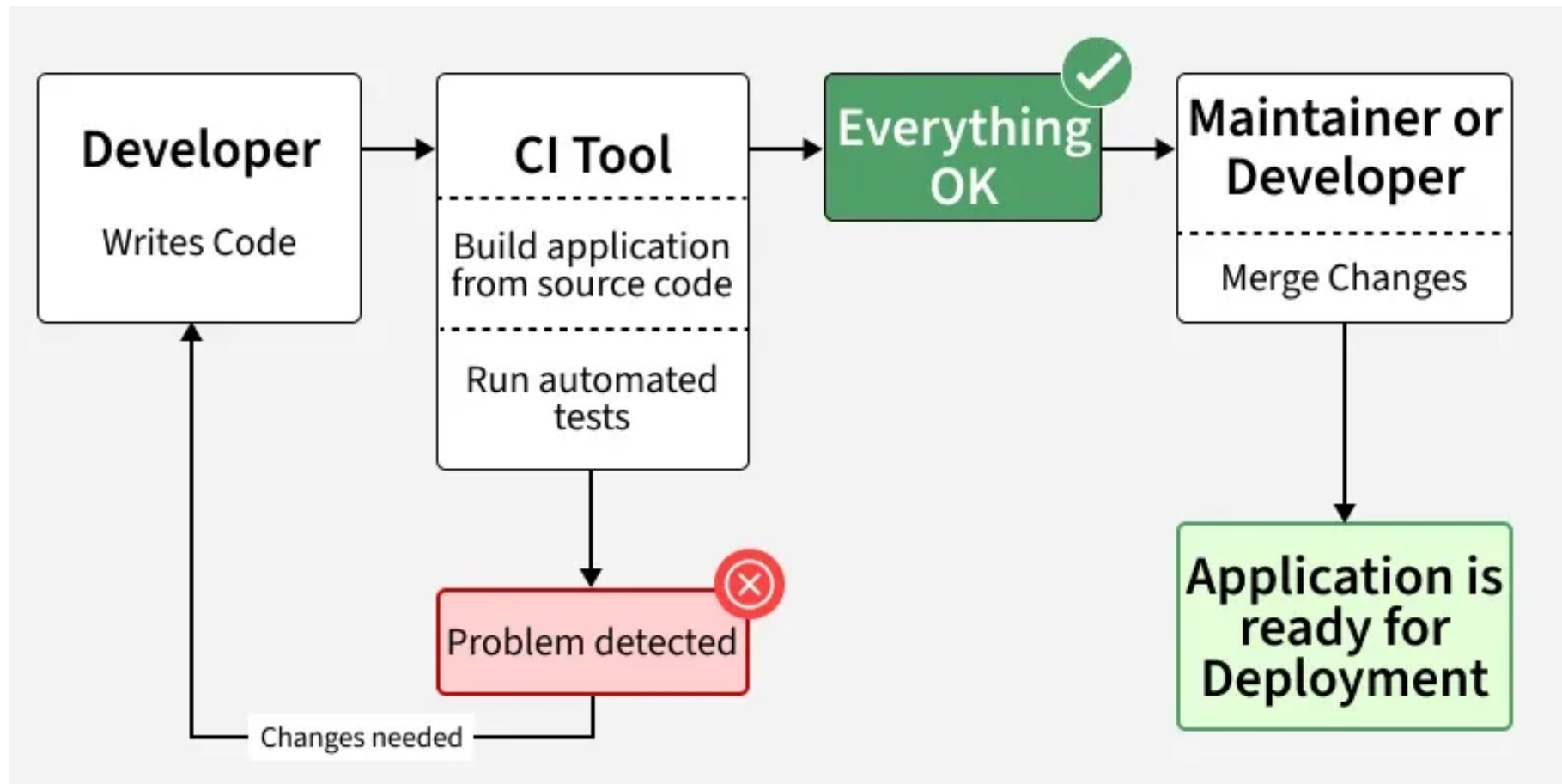


WHAT IS CI/CD?

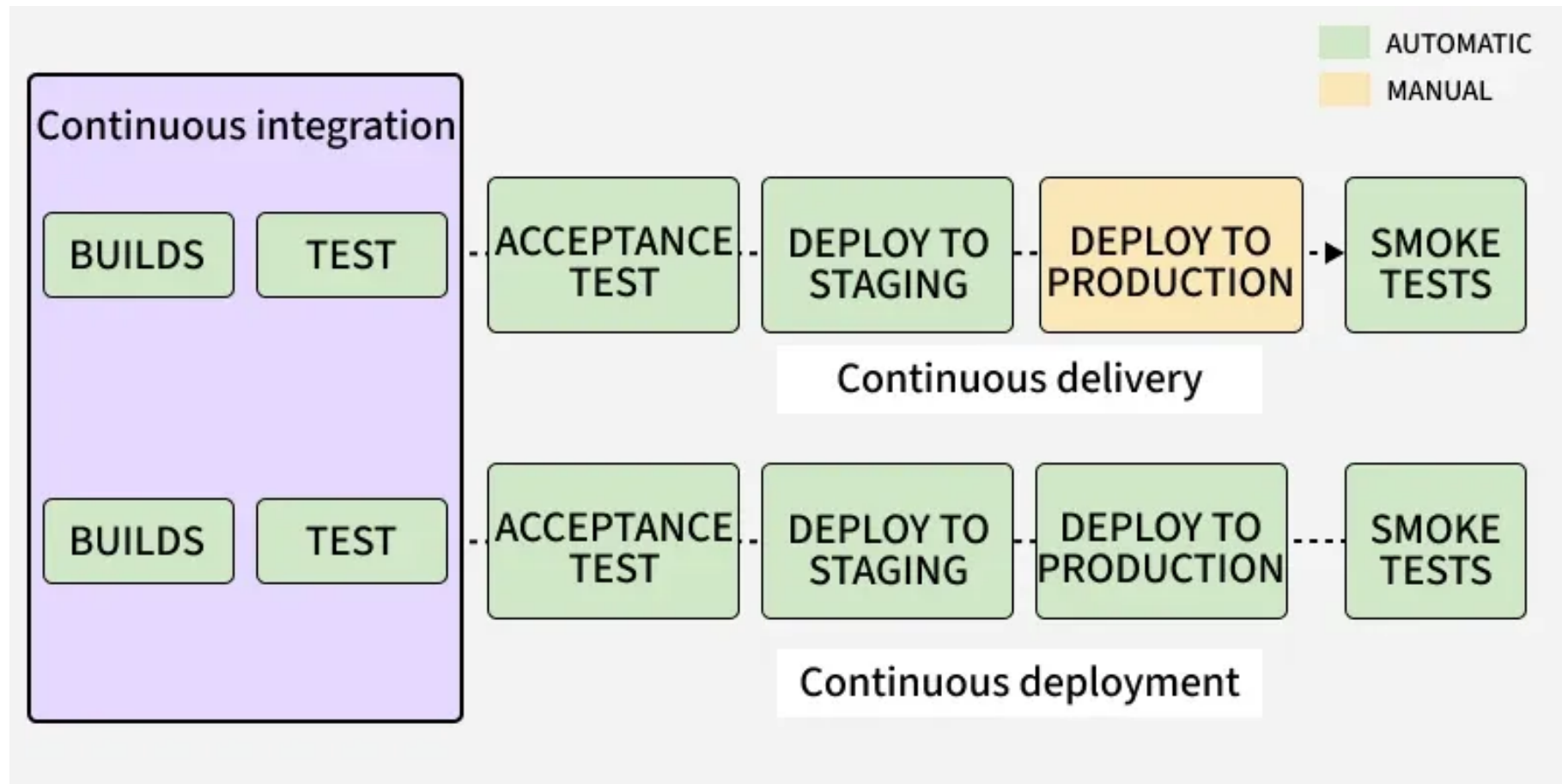
- **After CI/CD**
- With CI/CD, the whole process became faster, automated, and more reliable. Developers now commit their code frequently to a shared main branch, which prevents conflicts and ensures smooth integration.
- Automated pipelines run tests, builds, and checks as soon as code is pushed, so errors are caught early. Continuous Delivery makes sure that tested code is automatically packaged and ready for deployment
- While Continuous Deployment takes it a step further by directly releasing to production without manual steps. This means smaller, frequent updates instead of one big risky release.
- Tools like Git, Jenkins, Docker, and Kubernetes automate everything from building to deployment, making the workflow transparent and collaborative. Bugs are fixed quickly, rollbacks are easier, and overall software delivery happens in hours instead of days.



WHAT IS CI/CD?

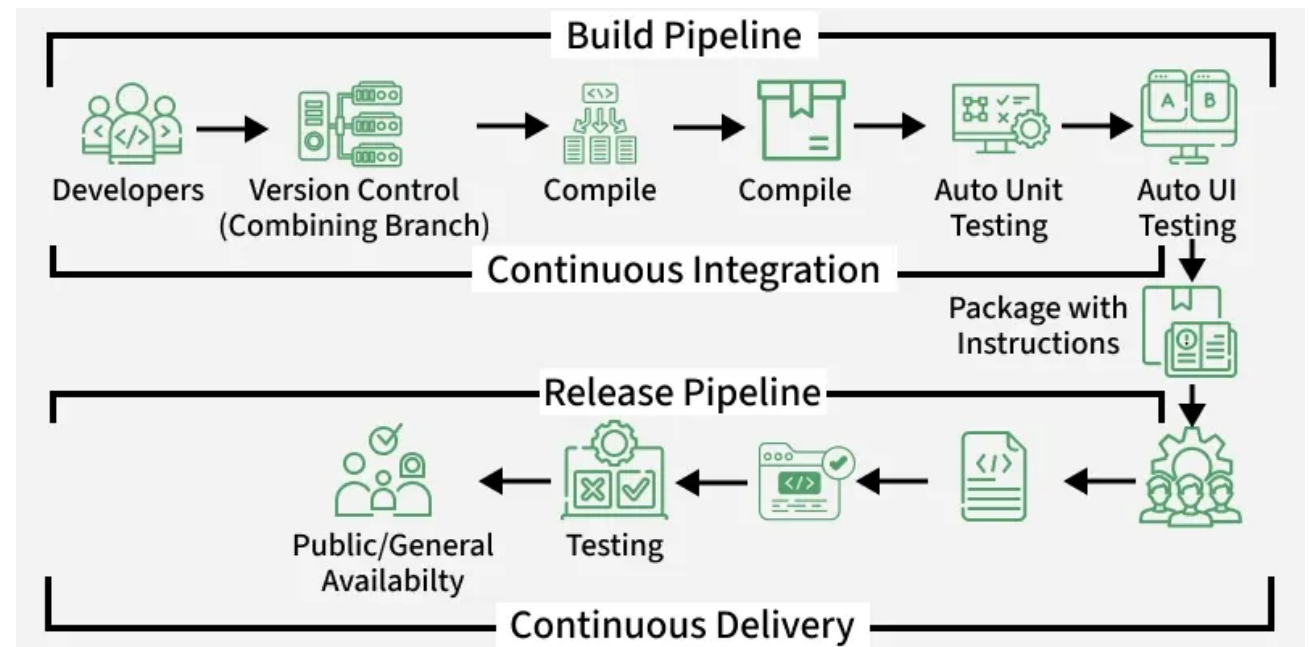


WHAT IS CI/CD?



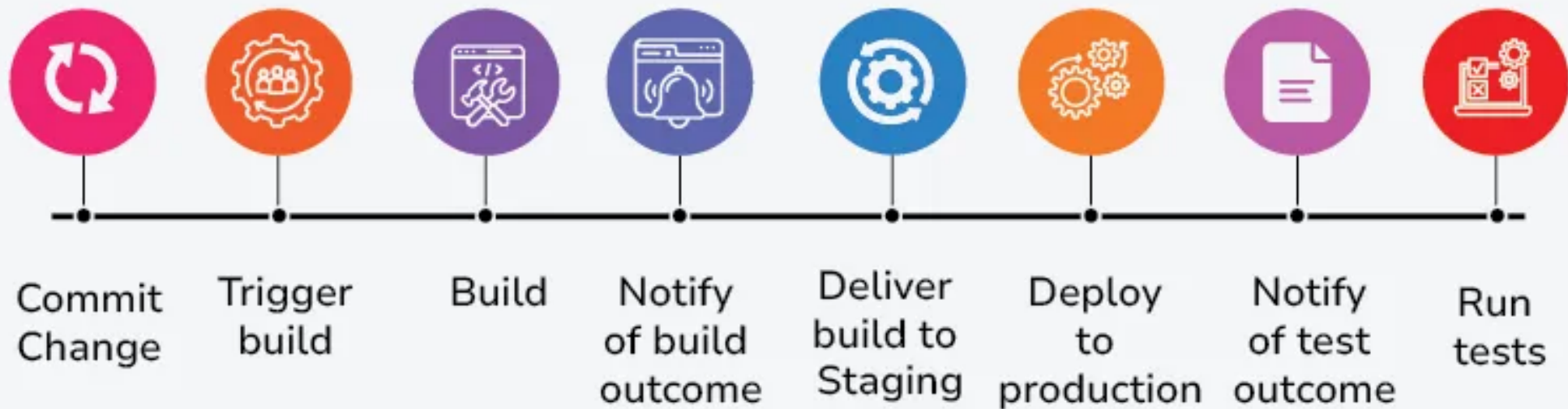
WHAT IS CI/CD?

- **What is CI/CD Pipeline?**
- Many people think CI/CD and CI/CD Pipeline are the same, but they are a bit different.
 - **CI/CD (Continuous Integration and Continuous Delivery):** This is a way of working where developers can make many code changes and deliver updates quickly.
 - **CI/CD Pipeline:** This is the actual step-by-step process that takes the code and automatically moves it from development to live production.
- In simple words, a CI/CD pipeline is a series of automatic steps that help developers build, test, and deliver code safely and quickly.



WHAT IS CI/CD?

CI/CD Pipeline Components



WHAT IS CI/CD?

1. Commit Change

- Developers make code changes and commit these changes to a version control system (e.g., Git). This step initiates the CI/CD pipeline. Committing code changes ensures that they are tracked and versioned properly.

2. Trigger Build

- The version control system detects the new commit and triggers the build process automatically. Automated triggering of the build process ensures that new changes are continuously integrated and tested.

3. Build

- The codebase is compiled and built into a deployable artifact, such as a binary executable or a Docker image. Common tools include Maven, Gradle, Ant for Java projects, and Docker for containerized applications. The build step verifies that the code compiles correctly and that all dependencies are resolved.

4. Notify of Build Outcome

- The CI/CD system notifies the team of the build results, whether it passed or failed. Immediate feedback on the build status helps developers quickly identify and resolve any build issues. Notification systems can include emails, chat integrations (like Slack or Microsoft Teams), or dashboards in tools like Jenkins or GitLab CI.

WHAT IS CI/CD?

5. Run Tests

- Automated tests are executed on the build artifact. These can include unit tests, integration tests, end-to-end tests, and more. Testing frameworks like JUnit, Selenium, TestNG, pytest, etc. Running tests ensures that the new code does not introduce any bugs or regressions and that it meets the required quality standards.

6. Notify of Test Outcome

- The results of the test suite are reported back to the development team. Quick feedback on test outcomes allows developers to address any failures promptly. Similar to build notifications, results can be sent via email, chat, or dashboards.

7. Deliver Build to Staging

- If tests pass, the build artifact is deployed to a staging environment. The staging environment simulates the production environment, allowing for final validation before production deployment. Deployment tools like Ansible, Chef, Puppet, Kubernetes, or cloud-specific services like AWS CodeDeploy.

8. Deploy to Production

- After successful validation in staging, the build is automatically or manually promoted to the production environment. This step makes the new features and fixes available to end-users. Deployment strategies might include blue-green deployment, canary releases, or rolling updates to minimize downtime and risk.

WHAT IS JENKINS?

- Jenkins is a free, open-source tool that helps developers automate the process of building, testing, and deploying code. It is written in Java and runs on the Java platform. By using Jenkins, we can make a continuous integration of projects(jobs) or end-to-end automation.
- **Core Concepts**
- The following are some important concepts of Jenkins:
 1. **Jenkins Controller (Formerly Master)**
- The Jenkins Controller serves as the central system for managing a Jenkins instance, often referred to as its "heart". It oversees agents and their connections, determining the tasks they should perform. Additionally, the Jenkins Controller loads plugins and ensures that jobs run in the correct sequence.

WHAT IS JENKINS?

2. Jenkins Agent (Formerly Slave)

- Jenkins Agent is a machine that performs tasks like running scripts, executing tests, or building components, etc. These tasks are assigned by the Jenkins Controller. Each agent can have its setup, like different operating systems, software, or hardware. This helps Jenkins handle many types of tasks and work faster by spreading the load.
- There are two main types of agents:
 - **Permanent Agents:** These are always ready and connected to Jenkins. They're like dedicated workers who are always on standby.
 - **Ephemeral Agents:** These are temporary. Jenkins starts them only when needed, usually in the cloud or using tools like Docker. When the job is done, they're shut down.

WHAT IS JENKINS?

3. Jenkins Node

- A Jenkins Node is a term that is used in Jenkins 2.0 to mean any system that can run Jenkins jobs. This is mainly used in Controllers and Agents and sometimes used instead of those terms. A node is a machine that Jenkins uses to run jobs like building and testing projects. If a node isn't performing well or its health drops below a certain threshold level, Jenkins will take that node offline to prevent any problems.

4. Jenkins Job

- Jenkins jobs are used to perform the work in the Jenkins system. A Jenkins job is an automated job that you set up in Jenkins to do things like build, test or deploy your code. Instead of doing these steps manually every time, Jenkins can do them for you whenever there's a change in your code. This helps save time and reduces mistakes.

WHAT IS JENKINS?

5. Jenkins Plugins

- Jenkins provides **2000+** community contributed Jenkins plugins which developers can use for building, deploying and automating any project. You can connect to GitHub, Slack, AWS, Docker, and more. You can easily install and upgrade these available plugins from the Jenkins Dashboard.

6. Jenkins Pipeline

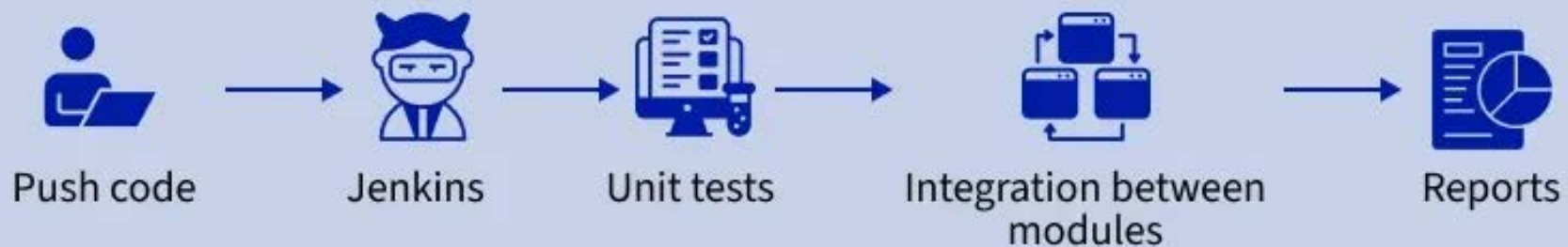
- DevOps professionals mostly work with pipelines because pipelines can automate processes like building, testing, and deploying the application. With the help of Continuous Integration / Continuous Deployment (CI/CD) Pipeline scripts we can automate the whole process which will increase productivity and save lots of time for the organization and can deliver quality applications to the end users.

WHAT IS JENKINS?

Delivering Mobile Apps from Developer Devices to Users



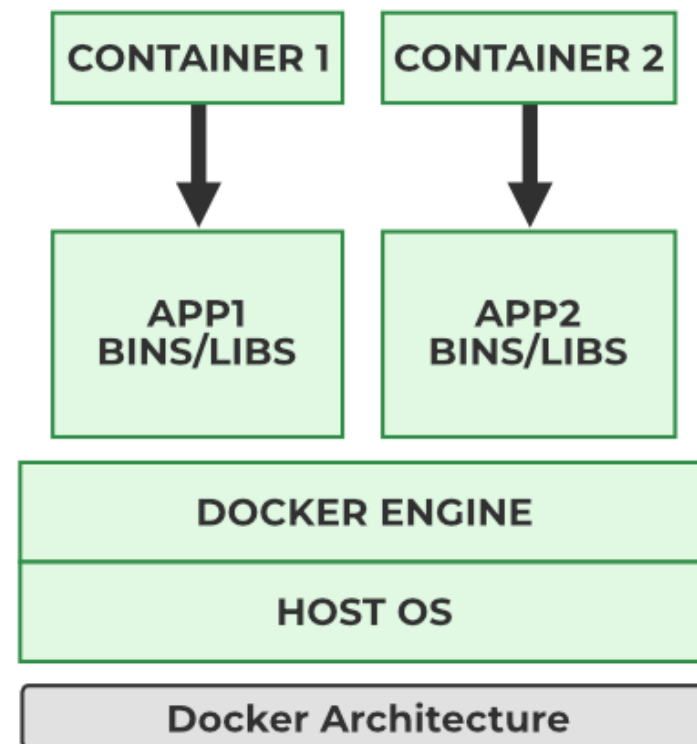
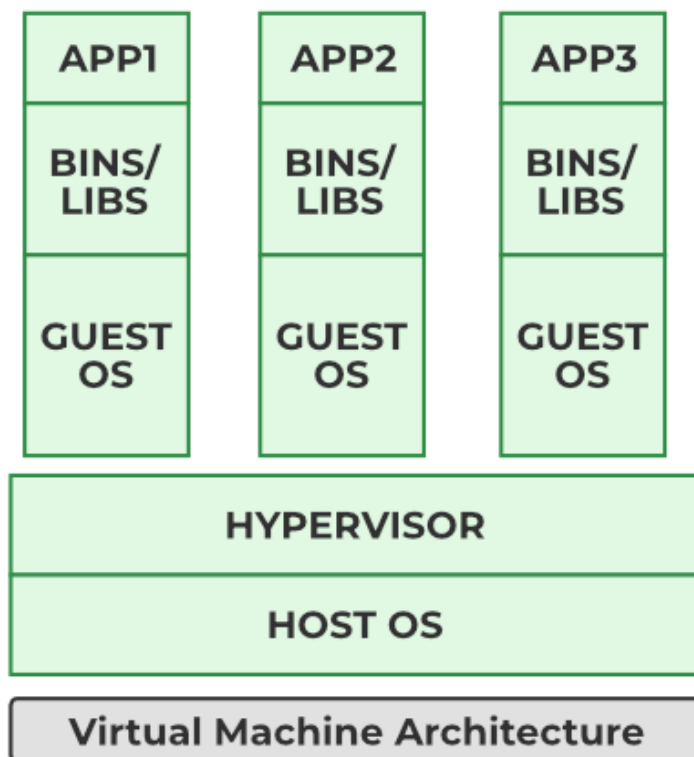
API Testing



WHAT IS DOCKER?

- Docker is an OS-level virtualization (or containerization) platform, which allows applications to share the host OS kernel instead of running a separate guest OS like in traditional virtualization. This design makes Docker containers lightweight, fast, and portable, while keeping them isolated from one another.
 - Written in the **Go** programming language.
 - Supports Windows, macOS, and Linux installations (Docker Engine runs natively on Linux).
 - Solves the “*works on my machine*” problem by ensuring code runs identically across environments.
 - Unlike VMware (hardware-level virtualization), Docker operates at the OS level.

WHAT IS DOCKER?



WHAT IS DOCKER?

- **Components of Docker**

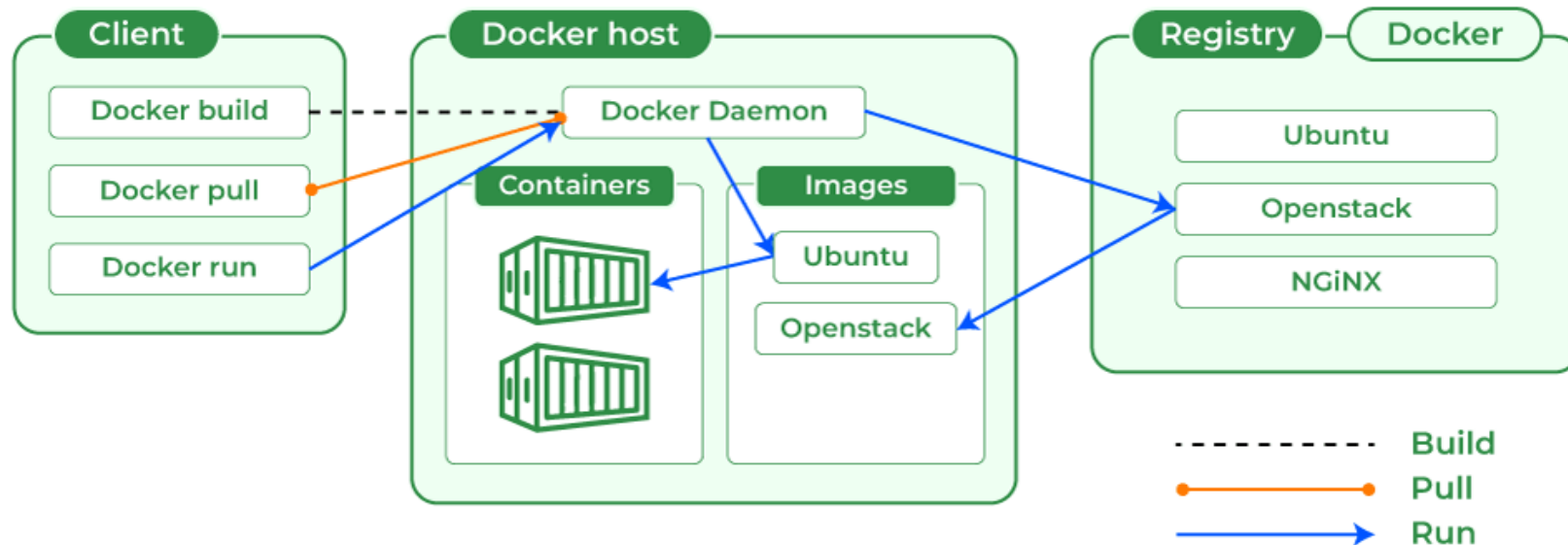
- The following are some of the key components of Docker:

- **Docker Engine:** Docker Engine has a core part docker daemon, that handles the creation and management of containers.
- **Docker Image:** Docker Image is a read-only template that is used for creating containers, containing the application code and dependencies.
- **Docker Hub:** It is a cloud based repository that is used for finding and sharing the container images.
- **Dockerfile:** It is a file that describes the steps to create an image quickly.
- **Docker Registry :** It is a storage distribution system for docker images, where you can store the images in both public and private modes.

WHAT IS DOCKER?

■ Docker Architecture and Working

- Docker makes use of a client-server architecture. The Docker client talks with the docker daemon which helps in building, running, and distributing the docker containers. The Docker client runs with the daemon on the same system or we can connect the Docker client with the Docker daemon remotely. With the help of REST API over a UNIX socket or a network, the docker client and daemon interact with each other.



WHAT IS KUBERNETES?

- Kubernetes, often shortened to K8s (K, 8 letters, s), is an open-source platform that automates the deployment, scaling, and management of containerized applications.
 - **Origin:** Developed by Google, inspired by internal systems Borg and Omega.
 - **Launch:** Officially released in 2014.
 - **CNCF Donation:** Donated to the Cloud Native Computing Foundation (CNCF) in 2015, which now maintains it.
 - **Adoption:** Widely used across major cloud providers today.
 - **Name Meaning:** *Kubernetes* comes from Greek, meaning "helmsman" or "pilot", symbolizing its role in steering applications.

WHAT IS KUBERNETES?

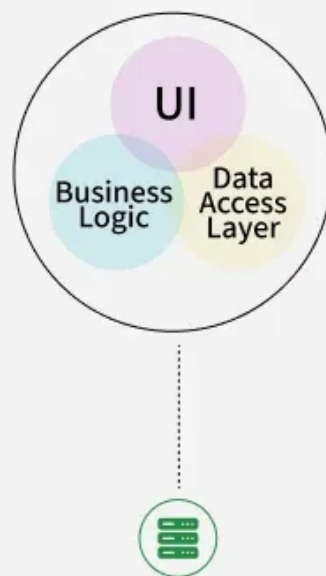
- **Some features of K8s:**

- **Automated Scheduling** – Efficiently places containers on nodes for optimal resource use.
- **Self-Healing** – Automatically restarts, replaces, and reschedules failed containers.
- **Rollouts & Rollbacks** – Manages application updates and reverts when needed.
- **Scaling & Load Balancing** – Supports horizontal scaling and distributes traffic.
- **Resource Optimization** – Monitors and ensures efficient resource utilization.

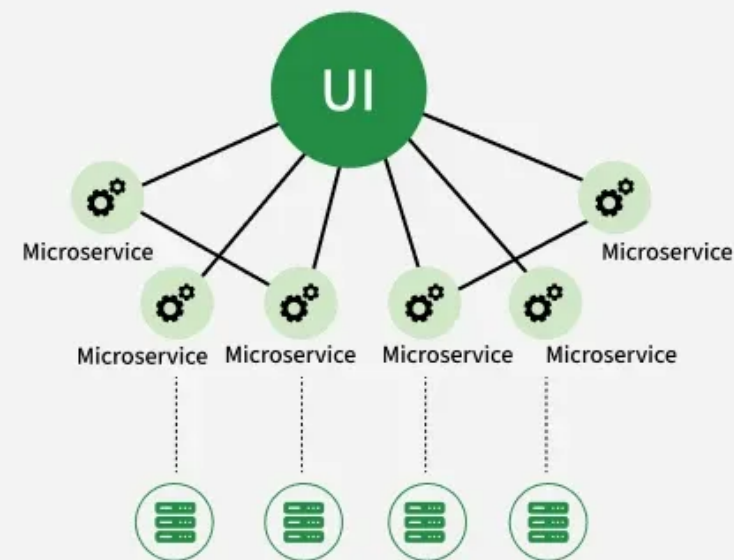
WHAT IS KUBERNETES?

- **Monolithic Vs Microservices**
- In the past, applications were built using a monolithic architecture, where everything was interconnected and bundled into one big codebase. This made updates risky, for example, if you wanted to change just the payment module in an e-commerce app, you had to redeploy the entire application. A small bug could crash the whole system.

Monolithic Architecture



Microservices Architecture



WHAT IS KUBERNETES?

- To overcome this, the industry moved toward microservices, where each feature (like payments, search, or notifications) is built and deployed independently. This made applications more flexible and scalable.
- However, the move to **microservices** introduced a new challenge. Instead of operating a single, monolithic application, organizations now had to manage hundreds or even thousands of small, containerized services. While containers effectively addressed packaging and portability, they created complexity at scale without a unified orchestration mechanism. Kubernetes emerged to solve this problem, providing an intelligent, automated platform for deploying, scaling, and coordinating large fleets of microservices.